

Build MiniTrack's frontend — **live.**

A finished design system on one side, a real API contract on the other. In eight prompted acts, we turn them into a complete, accessible React + Vite app — and let a reviewer agent and tests prove it's right.

• React + Vite + TypeScript

• Design system: "MiniTrack Precision"

• Plan → Implement → Review → Verify

Navigate with `←` `→` or `Space` · press `End` to jump to the wrap · the copy-paste prompts live in [RUNBOOK.md](#)

Five screens. Nothing invented.

MiniTrack is a minimalist task tracker. You connect with an API key, then view, create, edit, complete, and delete simple tasks.

`/connect`

Enter an API key (no accounts). Show/hide field, opt-in "remember this session," inline error.

`/tasks`

Filter All / Active / Completed, task cards, "Load more," inline complete & delete.

`/tasks/:id`

One task, its badges, and only the actions that exist.

`/tasks/new · edit`

One shared form for create and edit, with matching validation.

`*`

A friendly Not-Found with a way back.

The guardrail

Two inputs feed everything.

A design system, as CSS

Generated as "MiniTrack Precision" and handed to us finished — we consume it as-is.

- `DESIGN.md` — the written spec
- `tokens.css` — colors, type, spacing, radius, shadow
- `typography.css` — `.text-headline-*` utilities
- `components.css` — `.btn · .form-field · .badge · .task-card · .dialog`

A real backend contract

A FastAPI + SQLite JSON API — decoupled, CORS-enabled, key-protected.

- The `app/` layered backend (routes → services → data)
- The behavioral contract in `spec.md`
- Auth via an `X-API-Key` header
- **Four contract details** that will bite if ignored (slide 6)



THE TEACHING MOVE

We start from a delivered design system and wire React to it — using its global classes, not new CSS Modules.

Every act is one PIRV loop.

We don't make the smallest possible edit. We plan the slice, implement it in the right place, review it against the contract, and verify it runs.

P

Plan

Name the slice and the contract rules it must honor — before any code.

I

Implement

Prompt Claude to build it, with the gotchas baked into the prompt.

R

Review

A read-only `frontend-reviewer` agent checks it against `spec.md`.

V

Verify

Run it: dev server, typecheck, lint, and the Vitest suite.



WHY IT MATTERS HERE

The design system didn't appear by magic.

It was **generated** from a one-paragraph brief by a design tool — Google Stitch — and you can prototype the same brief in Figma Make. Same words, two tools; the tokens they produce are exactly what our CSS encodes.

Stitch / Figma Make – brief

```
Design "MiniTrack" – a minimalist task tracker.  
Corporate Minimalism: near-white bg, white  
surfaces, Indigo #4f46e5 primary, Inter, 4px  
spacing, 8px cards, pill badges (icon + label),  
3 soft elevation levels, centered 880px.  
# then: generate the 5 screens, invent no fields
```

brief



Stitch / Figma Make



tokens.css



THE POINT

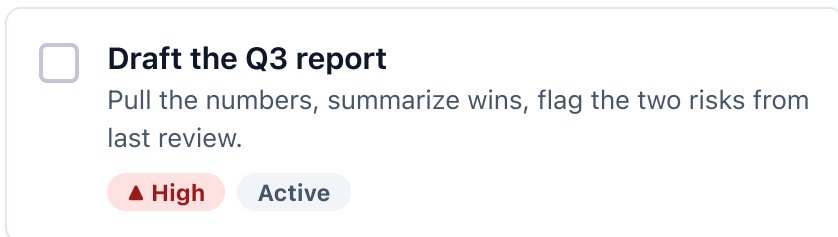
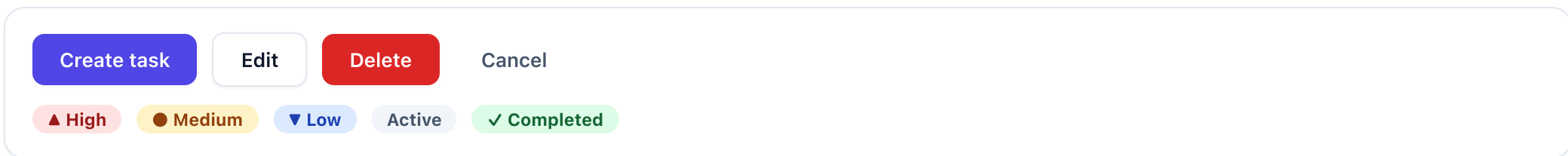
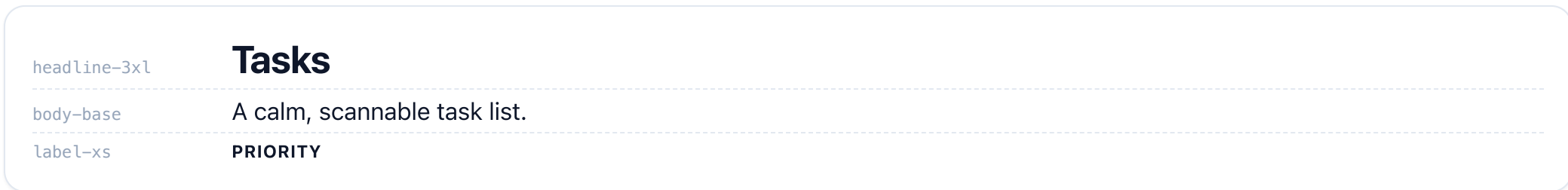
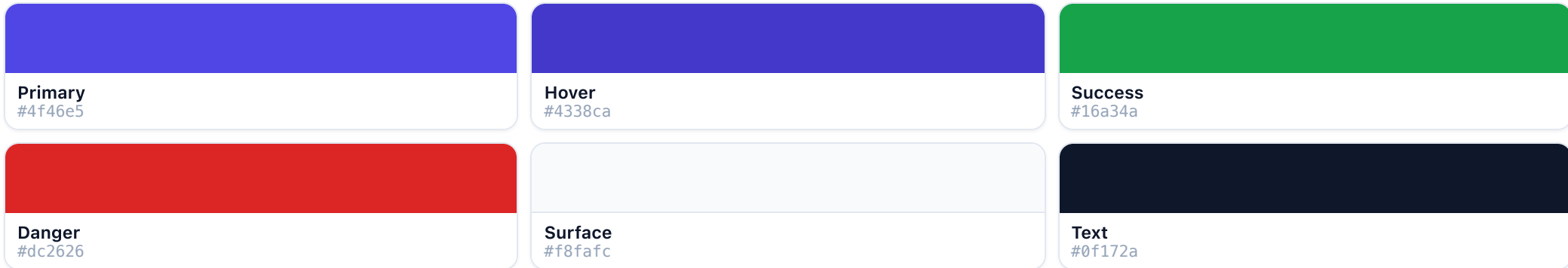
The prototype is a **picture to aim at**. The CSS in `src/styles/` — see `preview.html` — is what the app actually consumes in Acts 1–8.



GOTCHA

These tools will invent due dates / assignees if asked — resist it. And never paste a real API key into a design tool.

"MiniTrack Precision" — corporate minimalism.



Four details that bit the first build.

① 422 BODY

`detail` is a plain **string**, not an array. Never index `detail[0].msg`.

② PATCH /TASKS/{ID}

A **full replacement** — send the whole object; omitted fields reset; it can't change `completed`.

③ 401 BODY

Has **no** `request_id` (404/422 do). Treat it as optional.

④ CONNECT

Validate the key against GET `/tasks`, **not** `/health` (health ignores the key).

METHOD	PATH	NOTE
GET	<code>/tasks</code>	filter <code>?completed</code> , page <code>?limit&offset</code> — no total count
POST	<code>/tasks</code>	→ 201
PATCH	<code>/tasks/{id}</code>	full replacement
POST	<code>/tasks/{id}/complete</code>	one-way — no reopen endpoint
DELETE	<code>/tasks/{id}</code>	→ 204, needs a confirm dialog

ACT 1

Scaffold + Connect

ab8d9c6

ACT 2

Task list

9d01446

ACT 3

Create form

1eb8c15

ACT 4

Task detail

2aaf787

ACT 5

Edit + useFlash

7a30a7d

ACT 6

Mid-session 401

d313a63

ACT 7

Review pass

1a0bf22

ACT 8

Tests + verify

13ec334

ACT 1 Scaffold + design system + Connect

↔ ab8d9c6

Stand up Vite + React + TS in `frontend/`, wire in the delivered CSS, and build the API-key entry point. You *connect* — you don't *sign in*.

```
claude - minitrack_backend

# paste into Claude Code
Scaffold a Vite React+TS app in frontend/.
Copy DESIGN.md + src/styles/*.css into
frontend/src/styles and import index.css once.
Style with the design system's global classes
(.btn, .form-field) — no new CSS Modules.
Add router + ProtectedRoute + Layout, a
framework-agnostic fetch client (X-API-Key,
status:0 on network fail), ApiKeyContext, and a
/connect page that validates via GET /tasks.
```

- **Wire the design system:** one `index.css` import
- **Typed client:** no React import — unit-testable
- **Key safety:** in memory; `sessionStorage` opt-in only



GOTCHA

Validate against `GET /tasks`, not `/health` — health accepts any key.

✓ Verify — `npm run dev` → the Connect screen renders in Indigo; a wrong key is rejected, `demo-key-123` lands on `/tasks`.

ACT 2 Task list — filters & Load more

↔ 9d01446

The primary screen: URL-driven filters and pagination with *no total count*.

claude — frontend/

```
Build /tasks with a .task-card list and .badge
priority/status (icon + label, never color alone).
Filter All/Active/Completed in the URL (?status=).
Pagination is Load more: append, raise offset,
hide when a short page returns, reset on filter
change. Complete in place; Delete via a .dialog
confirm. loading / empty / error states, aria-live.
```

- **Filter state** lives in `?status=active`
- **Complete** → `POST /tasks/{id}/complete`
- **Delete** → accessible `<dialog>` → 204



GOTCHA

No total → **Load more**, hidden on a short page, reset on filter change. No numbered pages.

✓ Verify — filter chips change the URL & list; Complete moves a task; Delete asks first.

ACT 3 Create — shared form & validation

↔ 1eb8c15

A reusable `<TaskForm>` and validation that matches the backend exactly.

claude — frontend/

```
Build /tasks/new → POST /tasks (201). Extract a
reusable <TaskForm> + <FormField> on the .form-field
classes, shared with edit. Title required (trim;
block whitespace-only before any request). Priority
default medium. Disable submit in flight.
On 422, render the detail STRING inline —
never detail[0].msg.
```

- **Trim & block** whitespace-only titles client-side
- **No double POST** — disable while submitting
- **Reuse** the same form for edit



GOTCHA

422 `detail` is a **string**. `detail[0].msg` throws.

✓ Verify — a blank title is blocked before any call; a forced 422 renders its string inline.

ACT 4 Task detail — only real actions

↔ 2aaf787

Model the screen as explicit states, and expose only capabilities the API has.

claude — frontend/

```
Build /tasks/:taskId → GET /tasks/{id} as a
discriminated union: loading | loaded | notfound |
error. Show id, title, description, badges. Actions:
Edit, Complete (only when active), Delete
(confirm), Back. No "Reopen" action.
Unknown id → 404 → "Task not found" + link back.
```

- **State machine**, not scattered booleans
- **Complete** hides once done
- **404** is a first-class state



GOTCHA

No Reopen — completion is one-way; the backend has no un-complete endpoint.

✓ Verify — a completed task shows no Reopen; `/tasks/999999` → "Task not found."

ACT 5 Edit — the full-replacement PATCH

↔ 7a30a7d

The subtlest rule of the day, plus a small shared-hook refactor.

claude – frontend/

```
Build /tasks/:taskId/edit → PATCH /tasks/{id},  
reusing <TaskForm> in edit mode (load, pre-fill).  
PATCH is a FULL replacement: submit the  
entire TaskInput every time – never partial –  
and never send or touch completed.  
Extract a shared useFlash hook; reuse on list+detail.
```

- **Load first**, then pre-fill the fields
- **Whole object** on every save
- **useFlash** shared across screens



GOTCHA

Send only the changed field and PATCH silently **wipes the rest**. Never include completed.

✓ Verify — edit only the title → description & priority are preserved, not reset.

ACT 6 Mid-session 401 — reset the session

↔ d313a63

Handle a key that stops working *after* you connected — revoked or expired.

claude – frontend/

```
Add setUnauthorizedHandler to the client. On any
/tasks* 401 with a stored key: clear the key
(memory + sessionStorage), record a reason, and
redirect to /connect, which shows that reason.
Remember a 401 has no request_id – don't
rely on it here.
```

- **Global handler** in the client, not per-screen
- **Clear + redirect**, don't just show an error
- **Reason** surfaced on the Connect page



GOTCHA

Show an in-place error and the user is **stuck** on a screen that can't load. Reset the session.

✓ Verify — change the backend key mid-session → the next action bounces to `/connect` with the reason.

Close the loop — an agent, then tests.

ACT 7 Review · 1a0bf22

A read-only `frontend-reviewer` subagent checks the build against `spec.md`: invented features, wrong endpoints, partial PATCH, key exposure, missing states, accessibility.

- Move focus to "Tasks" + announce after actions
- Extract a shared `toDisplayError` helper
- Confirm `.env` is gitignored

ACT 8 Verify · 13ec334

A Vitest + RTL suite locks the load-bearing behaviors, plus a `MANUAL_TESTING.md` for the keyboard/screen-reader cases.

```
frontend/  
  
npm run typecheck && npm run lint && npm test  
✓ client · connect · load-more · detail · edit
```



GOTCHA

Native `<dialog>` needs a jsdom polyfill (`showModal/close`) or the dialog tests fail.

What they take away.

Not typing speed

The value was **front-loading the contract** so the build was right the first time.

The loop, not the code

Plan → Implement → **Review** (an agent) → **Verify** (tests) makes correctness repeatable.

Design as input

A delivered design system becomes a UI by **wiring components to its classes**.



TO EXTEND

Add a new field end-to-end — backend `spec.md` first, then type → client → form → tests — and watch the reviewer keep you honest.